

STUDENT ACADEMIC RECORD **DATABASE**

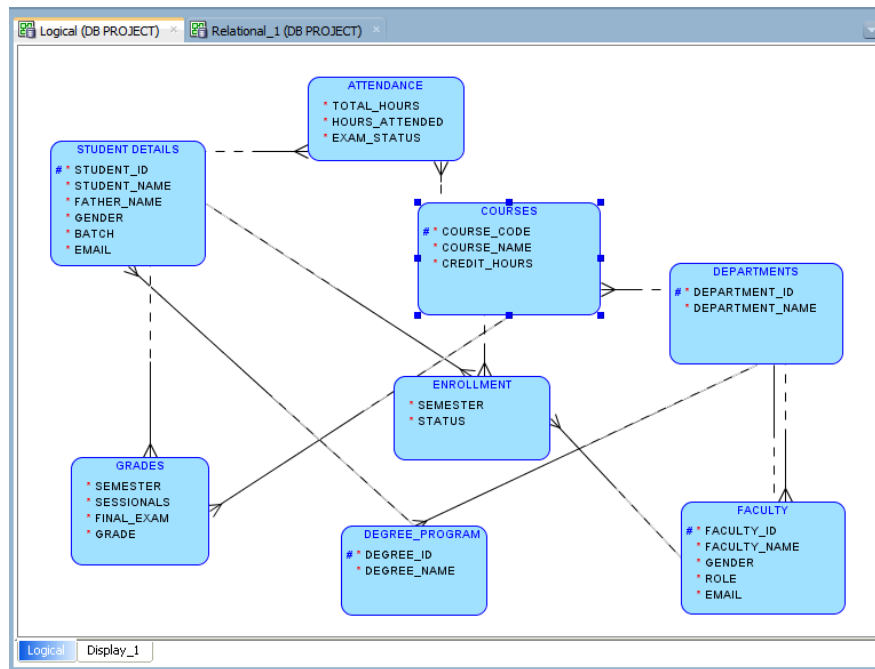
PROJECT DESCRIPTION

This project aims to create a centralized database system for educational institutions to manage student academic records, attendance, and performance analytics. The system will simplify administrative tasks and monitor student progress.

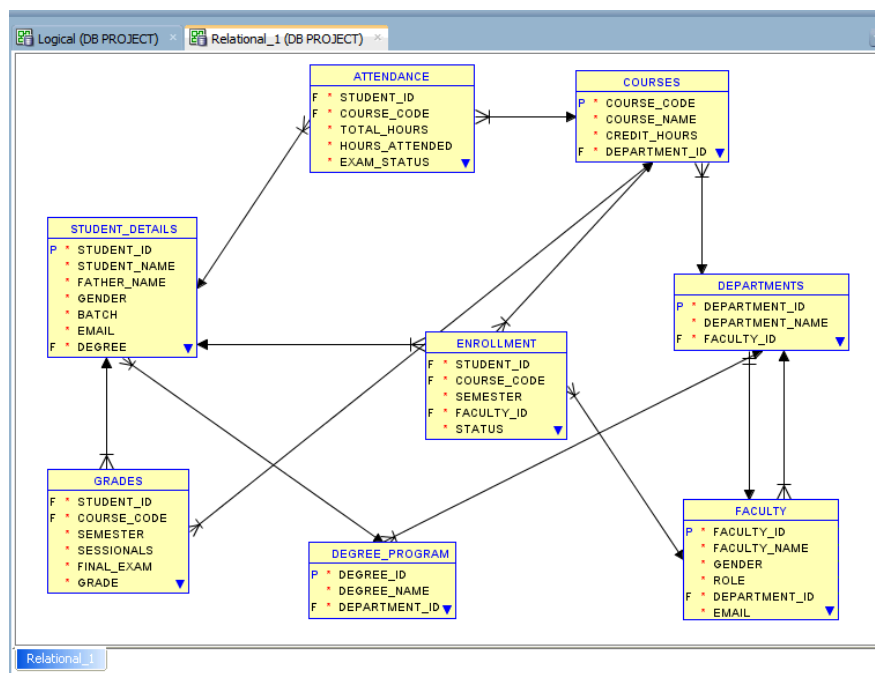
KEY FUNCTIONS:

- Student Records Management: Maintain comprehensive student details, including personal and enrollment information.
- Attendance Tracking: Calculate attendance percentages and to check if a student is eligible to appear in an exam.
- Grade Management: Record assignments, quizzes, exams, and calculate grades.
- Performance Analytics: Analyze trends in academic and attendance records to identify at-risk students.
- Faculty and Course Management: Manage faculty assignments and course details.

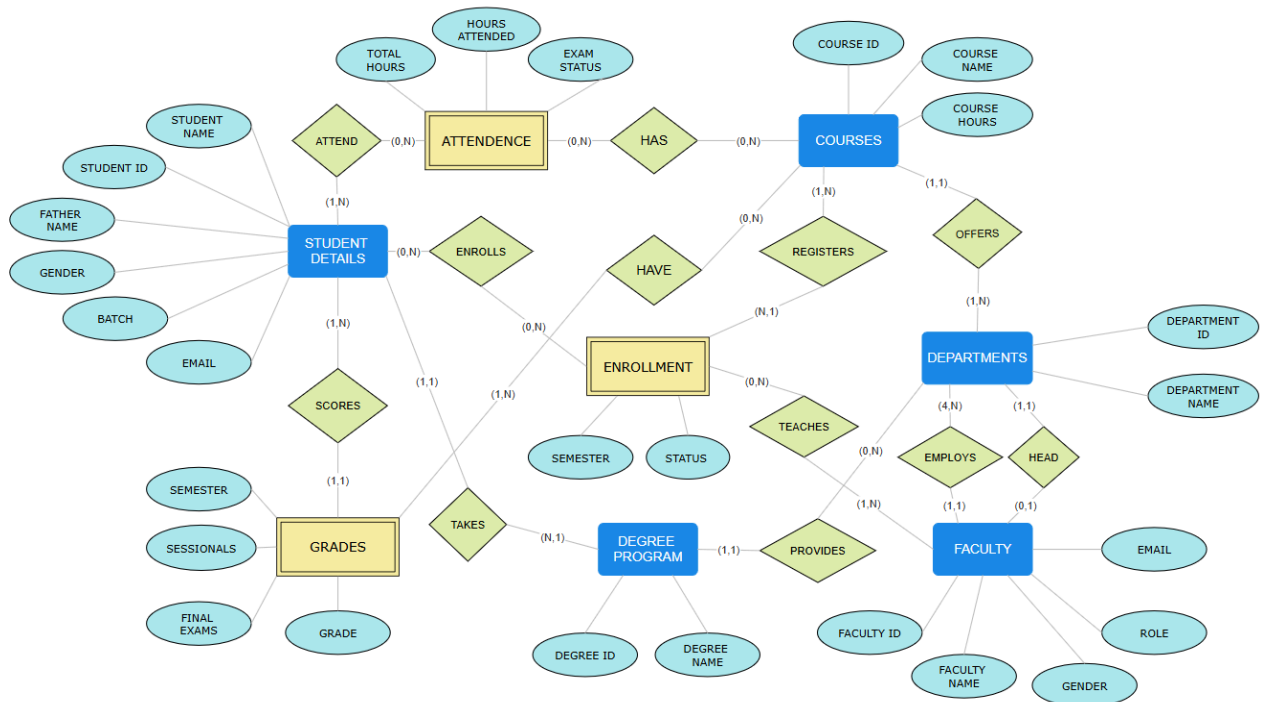
LOGICAL MODEL



RELATIONAL MODEL



ER MODEL



DESCRIPTION OF ER MODEL

1. STUDENT DETAILS

- **Purpose:** Stores information about the students.
- **Attributes:**
 - **STUDENT_ID:** Unique identifier for each student.
 - **STUDENT_NAME:** Full name of the student.
 - **FATHER_NAME:** Name of the student's father.
 - **GENDER:** Gender of the student.
 - **BATCH:** Academic batch the student belongs to.
 - **EMAIL:** Student's email address.
 - **DEGREE:** Degree program the student is enrolled in.
- **Relationships:**
 - The **DEGREE** attribute is a foreign key that references the **DEGREE_PROGRAM** table.

2. COURSES

- **Purpose:** Stores information about courses offered by the institution.
 - **Attributes:**
 - **COURSE_CODE:** Unique identifier for each course.
 - **COURSE_NAME:** Name of the course.
 - **CREDIT_HOURS:** The number of credit hours assigned to the course.
 - **DEPARTMENT_ID:** Identifier for the department offering the course.
 - **Relationships:**
 - **DEPARTMENT_ID** is a foreign key referencing the **DEPARTMENTS** table.
-

3. ATTENDANCE

- **Purpose:** Tracks student attendance in courses.
 - **Attributes:**
 - **STUDENT_ID:** Refers to the student (foreign key to **STUDENT_DETAILS**).
 - **COURSE_CODE:** Refers to the course (foreign key to **COURSES**).
 - **TOTAL_HOURS:** Total number of hours for the course.
 - **HOURS_ATTENDED:** Number of hours the student has attended.
 - **EXAM_STATUS:** Status of the student's exam (e.g., "Passed," "Failed").
 - **Relationships:**
 - Foreign keys linking to **STUDENT_DETAILS** (student) and **COURSES** (course).
-

4. ENROLLMENT

- **Purpose:** Tracks course enrollment details for students.
- **Attributes:**
 - **STUDENT_ID:** Refers to the student enrolling in the course.
 - **COURSE_CODE:** Refers to the course the student has enrolled in.
 - **SEMESTER:** The semester in which the course is taken.
 - **FACULTY_ID:** Refers to the faculty member teaching the course.
 - **STATUS:** The enrollment status (e.g., "Active," "Dropped").

- **Relationships:**
 - Foreign keys linking to **STUDENT_DETAILS** (student), **COURSES** (course), and **FACULTY** (faculty).
-

5. DEPARTMENTS

- **Purpose:** Stores information about the academic departments.
 - **Attributes:**
 - **DEPARTMENT_ID:** Unique identifier for each department.
 - **DEPARTMENT_NAME:** Name of the department.
 - **FACULTY_ID:** Refers to the faculty member leading the department.
 - **Relationships:**
 - **FACULTY_ID** is a foreign key referencing the **FACULTY** table.
 - **Additional Information:** A unique index is created on **FACULTY_ID** to ensure that each department has one faculty leader.
-

6. GRADES

- **Purpose:** Stores the grades of students for each course.
 - **Attributes:**
 - **STUDENT_ID:** Refers to the student (foreign key to **STUDENT_DETAILS**).
 - **COURSE_CODE:** Refers to the course (foreign key to **COURSES**).
 - **SEMESTER:** The semester in which the grades were awarded.
 - **SESSIONALS:** Marks or grade points for the sessional exams.
 - **FINAL_EXAM:** Marks or grade points for the final exam.
 - **GRADE:** The final grade awarded to the student (e.g., "A," "B").
 - **Relationships:**
 - Foreign keys linking to **STUDENT_DETAILS** (student) and **COURSES** (course).
-

7. DEGREE_PROGRAM

- **Purpose:** Stores information about different degree programs offered.
- **Attributes:**

- **DEGREE_ID**: Unique identifier for each degree program.
 - **DEGREE_NAME**: Name of the degree program (e.g., Bachelors in Computer Science).
 - **DEPARTMENT_ID**: Refers to the department offering the degree program.
 - **Relationships:**
 - **DEPARTMENT_ID** is a foreign key referencing the **DEPARTMENTS** table.
-

8. FACULTY

- **Purpose:** Stores information about faculty members.
 - **Attributes:**
 - **FACULTY_ID**: Unique identifier for each faculty member.
 - **FACULTY_NAME**: Name of the faculty member.
 - **GENDER**: Gender of the faculty member.
 - **ROLE**: Role of the faculty member (e.g., Professor, Lecturer).
 - **DEPARTMENT_ID**: Refers to the department to which the faculty member belongs.
 - **EMAIL**: Email address of the faculty member.
 - **Relationships:**
 - **DEPARTMENT_ID** is a foreign key referencing the **DEPARTMENTS** table.
-

Relationships Summary:

- **STUDENT_DETAILS** is linked to **DEGREE_PROGRAM** (student's degree program).
- **COURSES** are linked to **DEPARTMENTS** (course's department).
- **ATTENDANCE** connects **STUDENT_DETAILS** (students) and **COURSES** (courses).
- **ENROLLMENT** links **STUDENT_DETAILS**, **COURSES**, and **FACULTY** (who teaches the course).
- **DEPARTMENTS** are associated with **FACULTY** (department head).
- **GRADES** links **STUDENT_DETAILS** and **COURSES** (grading each student's performance).

DDL

```
CREATE TABLE STUDENT_DETAILS
```

```
(  
    STUDENT_ID VARCHAR2 (30) NOT NULL ,  
    STUDENT_NAME VARCHAR2 (30) NOT NULL ,  
    FATHER_NAME VARCHAR2 (30) NOT NULL ,  
    GENDER     VARCHAR2 (30) NOT NULL ,  
    BATCH      INTEGER NOT NULL ,  
    EMAIL      VARCHAR2 (30) NOT NULL ,  
    DEGREE     VARCHAR2 (30) NOT NULL  
);
```

```
ALTER TABLE STUDENT_DETAILS ADD CONSTRAINT STUDENT_DETAILS_PK  
PRIMARY KEY ( STUDENT_ID );
```

```
-- ERROR: FK name length exceeds maximum allowed length(30)
```

```
ALTER TABLE STUDENT_DETAILS ADD CONSTRAINT  
STUDENT_DETAILS_DEGREE_PROGRAM_FK FOREIGN KEY ( DEGREE )  
REFERENCES DEGREE_PROGRAM ( DEGREE_ID );
```

```
CREATE TABLE COURSES
```

```
(  
    COURSE_CODE VARCHAR2 (30) NOT NULL ,  
    COURSE_NAME  VARCHAR2 (30) NOT NULL ,  
    CREDIT_HOURS INTEGER NOT NULL ,  
    DEPARTMENT_ID VARCHAR2 (30) NOT NULL  
);
```

```
ALTER TABLE COURSES ADD CONSTRAINT COURSES_PK PRIMARY KEY (  
COURSE_CODE );
```

```
ALTER TABLE COURSES ADD CONSTRAINT COURSES_DEPARTMENTS_FK  
FOREIGN KEY ( DEPARTMENT_ID ) REFERENCES DEPARTMENTS (  
DEPARTMENT_ID );
```

```
CREATE TABLE ATTENDANCE
(
    STUDENT_ID   VARCHAR2 (30) NOT NULL ,
    COURSE_CODE  VARCHAR2 (30) NOT NULL ,
    TOTAL_HOURS  INTEGER NOT NULL ,
    HOURS_ATTENDED INTEGER NOT NULL ,
    EXAM_STATUS  VARCHAR2 (30) NOT NULL
);
ALTER TABLE ATTENDANCE ADD CONSTRAINT ATTENDANCE_COURSES_FK
FOREIGN KEY ( COURSE_CODE ) REFERENCES COURSES ( COURSE_CODE );
ALTER TABLE ATTENDANCE ADD CONSTRAINT
ATTENDANCE_STUDENT_DETAILS_FK FOREIGN KEY ( STUDENT_ID )
REFERENCES STUDENT_DETAILS ( STUDENT_ID );
```

```
CREATE TABLE ENROLLMENT
(
    STUDENT_ID   VARCHAR2 (30) NOT NULL ,
    COURSE_CODE  VARCHAR2 (30) NOT NULL ,
    SEMESTER     VARCHAR2 (30) NOT NULL ,
    FACULTY_ID   VARCHAR2 (30) NOT NULL ,
    STATUS       VARCHAR2 (30) NOT NULL
);
ALTER TABLE ENROLLMENT ADD CONSTRAINT ENROLLMENT_COURSES_FK
FOREIGN KEY ( COURSE_CODE ) REFERENCES COURSES ( COURSE_CODE );
ALTER TABLE ENROLLMENT ADD CONSTRAINT ENROLLMENT_FACULTY_FK
FOREIGN KEY ( FACULTY_ID ) REFERENCES FACULTY ( FACULTY_ID );
ALTER TABLE ENROLLMENT ADD CONSTRAINT
ENROLLMENT_STUDENT_DETAILS_FK FOREIGN KEY ( STUDENT_ID )
REFERENCES STUDENT_DETAILS ( STUDENT_ID );
```

```
CREATE TABLE DEPARTMENTS
(
    DEPARTMENT_ID VARCHAR2 (30) NOT NULL ,
    DEPARTMENT_NAME VARCHAR2 (30) NOT NULL ,
    HOD_ID    VARCHAR2 (30) NOT NULL
);
CREATE UNIQUE INDEX DEPARTMENTS__IDX ON DEPARTMENTS
(
    FACULTY_ID ASC
)
;
ALTER TABLE DEPARTMENTS ADD CONSTRAINT DEPARTMENTS_PK PRIMARY
KEY ( DEPARTMENT_ID );
ALTER TABLE DEPARTMENTS ADD CONSTRAINT DEPARTMENTS_FACULTY_FK
FOREIGN KEY (FACULTY_ID ) REFERENCES FACULTY ( FACULTY_ID );
ALTER TABLE DEPARTMENTS RENAME COLUMN FACULTY_ID TO HOD_ID;
```

```
CREATE TABLE GRADES
(
    STUDENT_ID VARCHAR2 (30) NOT NULL ,
    COURSE_CODE VARCHAR2 (30) NOT NULL ,
    SEMESTER    VARCHAR2 (30) NOT NULL ,
    SESSIONALS  INTEGER NOT NULL ,
    FINAL_EXAM  INTEGER NOT NULL ,
    GRADE       VARCHAR2 (30) NOT NULL
);
ALTER TABLE GRADES ADD CONSTRAINT GRADES_COURSES_FK FOREIGN
KEY ( COURSE_CODE ) REFERENCES COURSES ( COURSE_CODE );
ALTER TABLE GRADES ADD CONSTRAINT GRADES_STUDENT_DETAILS_FK
FOREIGN KEY ( STUDENT_ID ) REFERENCES STUDENT_DETAILS ( STUDENT_ID
);
```

```
CREATE TABLE DEGREE_PROGRAM
(
    DEGREE_ID    VARCHAR2 (30) NOT NULL ,
    DEGREE_NAME  VARCHAR2 (30) NOT NULL ,
    DEPARTMENT_ID VARCHAR2 (30) NOT NULL
);
ALTER TABLE DEGREE_PROGRAM ADD CONSTRAINT DEGREE_PROGRAM_PK
PRIMARY KEY ( DEGREE_ID );
ALTER TABLE DEGREE_PROGRAM ADD CONSTRAINT
DEGREE_PROGRAM_DEPARTMENTS_FK FOREIGN KEY ( DEPARTMENT_ID )
REFERENCES DEPARTMENTS ( DEPARTMENT_ID );
```

```
CREATE TABLE FACULTY
(
    FACULTY_ID    VARCHAR2 (30) NOT NULL ,
    FACULTY_NAME  VARCHAR2 (30) NOT NULL ,
    GENDER        VARCHAR2 (30) NOT NULL ,
    ROLE          VARCHAR2 (30) NOT NULL ,
    DEPARTMENT_ID VARCHAR2 (30) NOT NULL ,
    EMAIL         VARCHAR2 (30) NOT NULL
);
ALTER TABLE FACULTY ADD CONSTRAINT FACULTY_PK PRIMARY KEY (
FACULTY_ID );
ALTER TABLE FACULTY ADD CONSTRAINT FACULTY_DEPARTMENTS_FK
FOREIGN KEY ( DEPARTMENT_ID ) REFERENCES DEPARTMENTS (
DEPARTMENT_ID );
```

DML

INSERT QUERIES:

```
INSERT INTO STUDENT_DETAILS (STUDENT_ID, STUDENT_NAME,  
FATHER_NAME, GENDER, BATCH, EMAIL, DEGREE)  
VALUES  
( 'S001', 'John Doe', 'Michael Doe', 'Male', 2023, 'john.doe@email.com', 'BSc_CS'),  
( 'S002', 'Jane Smith', 'James Smith', 'Female', 2023, 'jane.smith@email.com',  
'BSc_CS'),  
( 'S003', 'Sam Brown', 'William Brown', 'Male', 2023, 'sam.brown@email.com',  
'BSc_Physics'),  
( 'S004', 'Emily White', 'Robert White', 'Female', 2023, 'emily.white@email.com',  
'BSc_Chemistry'),  
( 'S005', 'Michael Green', 'Joseph Green', 'Male', 2023, 'michael.green@email.com',  
'BSc_Biology'),  
( 'S006', 'Olivia Black', 'Thomas Black', 'Female', 2023, 'olivia.black@email.com',  
'BSc_Math'),  
( 'S007', 'Daniel Harris', 'Edward Harris', 'Male', 2023, 'daniel.harris@email.com',  
'BSc_CS'),  
( 'S008', 'Sophia Clark', 'David Clark', 'Female', 2023, 'sophia.clark@email.com',  
'BSc_CS'),  
( 'S009', 'David Lewis', 'Paul Lewis', 'Male', 2023, 'david.lewis@email.com',  
'BSc_Physics'),  
( 'S010', 'Emma Turner', 'Brian Turner', 'Female', 2023, 'emma.turner@email.com',  
'BSc_Chemistry');
```

```
INSERT INTO DEGREE_PROGRAM (DEGREE_ID, DEGREE_NAME,  
DEPARTMENT_ID)  
VALUES  
( 'BSc_CS', 'Computer Science', 'CS'),  
( 'BSc_Physics', 'Physics', 'PH'),  
( 'BSc_Chemistry', 'Chemistry', 'CH'),  
( 'BSc_Biology', 'Biology', 'BI'),  
( 'BSc_Math', 'Mathematics', 'MA');
```

```
INSERT INTO FACULTY (FACULTY_ID, FACULTY_NAME, GENDER, ROLE,
DEPARTMENT_ID, EMAIL)
VALUES
('F001', 'Dr. Alice Williams', 'Female', 'Professor', 'CS', 'alice.williams@email.com'),
('F002', 'Dr. Bob Johnson', 'Male', 'Assistant Professor', 'CS', 'bob.johnson@email.com'),
('F003', 'Dr. Carol Davis', 'Female', 'Professor', 'PH', 'carol.davis@email.com'),
('F004', 'Dr. Daniel Lee', 'Male', 'Associate Professor', 'CH', 'daniel.lee@email.com'),
('F005', 'Dr. Eva Moore', 'Female', 'Professor', 'BI', 'eva.moore@email.com'),
('F006', 'Dr. Frank White', 'Male', 'Lecturer', 'MA', 'frank.white@email.com');
```

```
INSERT INTO DEPARTMENTS (DEPARTMENT_ID, DEPARTMENT_NAME,
FACULTY_ID)
VALUES
('CS', 'Computer Science', 'F001'),
('PH', 'Physics', 'F003'),
('CH', 'Chemistry', 'F004'),
('BI', 'Biology', 'F005'),
('MA', 'Mathematics', 'F006');
```

```
INSERT INTO COURSES (COURSE_CODE, COURSE_NAME, CREDIT_HOURS,
DEPARTMENT_ID)
VALUES
('CS101', 'Introduction to Computer Science', 3, 'CS'),
('CS102', 'Data Structures', 4, 'CS'),
('PH101', 'Introduction to Physics', 3, 'PH'),
('CH101', 'Chemistry 101', 3, 'CH'),
('BIO101', 'Biology 101', 3, 'BI'),
('MA101', 'Mathematics I', 4, 'MA'),
('CS201', 'Algorithms', 4, 'CS'),
('PH102', 'Advanced Physics', 4, 'PH'),
```

```
('CH102', 'Organic Chemistry', 4, 'CH'),  
( 'BIO102', 'Ecology', 3, 'BI');
```

```
INSERT INTO GRADES (STUDENT_ID, COURSE_CODE, SEMESTER,  
SESSIONALS, FINAL_EXAM, GRADE)
```

```
VALUES
```

```
('S001', 'CS101', 'Fall 2023', 80, 90, 'A'),  
( 'S002', 'CS102', 'Fall 2023', 75, 85, 'B'),  
( 'S003', 'PH101', 'Fall 2023', 60, 50, 'C'),  
( 'S004', 'CH101', 'Fall 2023', 95, 90, 'A'),  
( 'S005', 'BIO101', 'Fall 2023', 70, 80, 'B'),  
( 'S006', 'MA101', 'Fall 2023', 65, 55, 'C'),  
( 'S007', 'CS101', 'Fall 2023', 85, 90, 'A'),  
( 'S008', 'CS102', 'Fall 2023', 88, 85, 'A'),  
( 'S009', 'PH102', 'Fall 2023', 75, 85, 'B'),  
( 'S010', 'CH102', 'Fall 2023', 80, 88, 'A');
```

```
INSERT INTO ATTENDANCE (STUDENT_ID, COURSE_CODE, TOTAL_HOURS,  
HOURS_ATTENDED, EXAM_STATUS)
```

```
VALUES
```

```
('S001', 'CS101', 40, 35, 'ELIGIBLE'),  
( 'S002', 'CS102', 40, 38, 'ELIGIBLE'),  
( 'S003', 'PH101', 40, 30, 'INELIGIBLE'),  
( 'S004', 'CH101', 40, 40, 'ELIGIBLE'),  
( 'S005', 'BIO101', 40, 37, 'ELIGIBLE'),  
( 'S006', 'MA101', 40, 32, 'INELIGIBLE'),  
( 'S007', 'CS101', 40, 40, 'ELIGIBLE'),  
( 'S008', 'CS102', 40, 35, 'ELIGIBLE'),  
( 'S009', 'PH102', 40, 38, 'ELIGIBLE'),  
( 'S010', 'CH102', 40, 40, 'ELIGIBLE');
```

```
INSERT INTO ENROLLMENT (STUDENT_ID, COURSE_CODE, SEMESTER,  
FACULTY_ID, STATUS)
```

```
VALUES
```

('S001', 'CS101', 'Fall 2023', 'F001', 'Enrolled'),
('S002', 'CS102', 'Fall 2023', 'F002', 'Enrolled'),
('S003', 'PH101', 'Fall 2023', 'F003', 'Enrolled'),
('S004', 'CH101', 'Fall 2023', 'F004', 'Enrolled'),
('S005', 'BIO101', 'Fall 2023', 'F005', 'Enrolled'),
('S006', 'MA101', 'Fall 2023', 'F006', 'Enrolled'),
('S007', 'CS101', 'Fall 2023', 'F001', 'Enrolled'),
('S008', 'CS102', 'Fall 2023', 'F002', 'Enrolled'),
('S009', 'PH102', 'Fall 2023', 'F003', 'Enrolled'),
('S010', 'CH102', 'Fall 2023', 'F004', 'Enrolled');

UPDATE QUERIES:

UPDATE STUDENT_DETAILS
SET EMAIL = 'john.doe_updated@email.com'
WHERE STUDENT_ID = 'S001';

UPDATE DEGREE_PROGRAM
SET DEGREE_NAME = 'Computer Science'
WHERE DEGREE_ID = 'BSc_CS';

UPDATE FACULTY
SET ROLE = 'Head of Department'
WHERE FACULTY_ID = 'F001';

UPDATE DEPARTMENTS
SET DEPARTMENT_NAME = 'Department of Computer Science'
WHERE DEPARTMENT_ID = 'CS';

UPDATE COURSES
SET CREDIT_HOURS = 4
WHERE COURSE_CODE = 'CS101';

UPDATE GRADES
SET GRADE = 'B'
WHERE STUDENT_ID = 'S003' AND COURSE_CODE = 'PH101';

UPDATE ATTENDANCE

```
SET EXAM_STATUS = 'Passed'
WHERE STUDENT_ID = 'S009' AND COURSE_CODE = 'PH102';
```

```
UPDATE ENROLLMENT
SET STATUS = 'Completed'
WHERE STUDENT_ID = 'S010' AND COURSE_CODE = 'CH102';
```

```
UPDATE FACULTY SET ROLE = 'Lecturer' WHERE FACULTY_ID = 'F001';
```

DELETE QUERIES:

```
DELETE FROM GRADES
WHERE STUDENT_ID = 'S003' AND COURSE_CODE = 'PH101';
```

```
DELETE FROM ATTENDANCE
WHERE STUDENT_ID = 'S009' AND COURSE_CODE = 'PH102';
```

```
DELETE FROM ENROLLMENT
WHERE STUDENT_ID = 'S010' AND COURSE_CODE = 'CH102';
```

TRANSACTION HANDLING

```
BEGIN
    INSERT INTO STUDENT_DETAILS
        (STUDENT_ID, STUDENT_NAME, FATHER_NAME, GENDER, BATCH, EMAIL,
        DEGREE)
    VALUES
        ('S011', 'James Doe', 'John Doe', 'Male', 2023, 'james.doe@email.com', 'BSc_CS');
    INSERT INTO ENROLLMENT
        (STUDENT_ID, COURSE_CODE, SEMESTER, FACULTY_ID, STATUS)
    VALUES
        ('S011', 'CS101', 'Fall 2023', 'F001', 'Enrolled');
    INSERT INTO GRADES
        (STUDENT_ID, COURSE_CODE, SEMESTER, SESSIONALS, FINAL_EXAM, GRADE)
    VALUES
        ('S011', 'CS101', 'Fall 2023', 45, 33, 'B');
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
END;
```

```
BEGIN
  UPDATE GRADES
  SET GRADE =
    CASE
      WHEN (SESSIONALS + FINAL_EXAM) > 86 THEN 'A'
      WHEN (SESSIONALS + FINAL_EXAM) > 74 THEN 'B'
      WHEN (SESSIONALS + FINAL_EXAM) > 62 THEN 'C'
      WHEN (SESSIONALS + FINAL_EXAM) > 50 THEN 'D'
      ELSE 'F'
    END;
  COMMIT;

EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

```
BEGIN
  DELETE FROM STUDENT_DETAILS
  WHERE STUDENT_ID = 'S011';
  COMMIT;

EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

```
BEGIN
  UPDATE ATTENDANCE
  SET EXAM_STATUS =
    CASE
      WHEN (HOURS_ATTENDED/TOTAL_HOURS > 0.80) THEN 'ELIGIBLE'
      ELSE 'INELIGIBLE'
    END;
  COMMIT;
END;
```

DATA

ATTENDANCE TABLE:

	STUDENT_ID	COURSE_CODE	TOTAL_HOURS	HOURS_ATTENDED	EXAM_STATUS
1	S001	CS101	40	35	ELIGIBLE
2	S002	CS102	40	38	ELIGIBLE
3	S003	PH101	40	30	INELIGIBLE
4	S004	CH101	40	40	ELIGIBLE
5	S005	BIO101	40	37	ELIGIBLE
6	S006	MA101	40	32	INELIGIBLE
7	S007	CS101	40	40	ELIGIBLE
8	S008	CS102	40	35	ELIGIBLE
9	S009	PH102	40	38	ELIGIBLE

COURSES TABLE:

	COURSE_CODE	COURSE_NAME	CREDIT_HOURS	DEPARTMENT_ID
1	CS101	Computer Science	4	CS
2	CS102	Data Structures	4	CS
3	PH101	Introduction to Physics	3	PH
4	CH101	Chemistry 101	3	CH
5	BIO101	Biology 101	3	BI
6	MA101	Mathematics I	4	MA
7	CS201	Algorithms	4	CS
8	PH102	Advanced Physics	4	PH
9	CH102	Organic Chemistry	4	CH
10	BIO102	Ecology	3	BI

DEGREE_PROGRAM TABLE:

	DEGREE_ID	DEGREE_NAME	DEPARTMENT_ID
1	BSc_CS	Computer Science	CS
2	BSc_Physics	Physics	PH
3	BSc_Chemistry	Chemistry	CH
4	BSc_Biology	Biology	BI
5	BSc_Math	Mathematics	MA

DEPARTMENTS TABLE:

	DEPARTMENT_ID	DEPARTMENT_NAME	HOD_ID
1	CS	Computer Science	F001
2	PH	Physics	F003
3	CH	Chemistry	F004
4	BI	Biology	F005
5	MA	Mathematics	F006

ENROLLMENT TABLE:

	STUDENT_ID	COURSE_CODE	SEMESTER	FACULTY_ID	STATUS
1	S001	CS101	Fall 2023	F001	Enrolled
2	S002	CS102	Fall 2023	F002	Enrolled
3	S003	PH101	Fall 2023	F003	Enrolled
4	S004	CH101	Fall 2023	F004	Enrolled
5	S005	BIO101	Fall 2023	F005	Enrolled
6	S006	MA101	Fall 2023	F006	Enrolled
7	S007	CS101	Fall 2023	F001	Enrolled
8	S008	CS102	Fall 2023	F002	Enrolled
9	S009	PH102	Fall 2023	F003	Enrolled
10	S011	CS101	Fall 2023	F001	Enrolled

GRADES TABLE:

	STUDENT_ID	COURSE_CODE	SEMESTER	SESSIONALS	FINAL_EXAM	GRADE
1	S001	CS101	Fall 2023	40	45	B
2	S002	CS102	Fall 2023	38	43	B
3	S004	CH101	Fall 2023	48	45	A
4	S005	BIO101	Fall 2023	35	40	B
5	S006	MA101	Fall 2023	33	28	D
6	S006	MA101	Fall 2023	33	28	D
7	S007	CS101	Fall 2023	43	45	A
8	S008	CS102	Fall 2023	44	43	A
9	S009	PH102	Fall 2023	38	43	B
10	S010	CH102	Fall 2023	40	44	B
11	S011	CS101	Fall 2023	23	17	F

STUDENT_DETAILS:

	STUDENT_ID	STUDENT_NAME	FATHER_NAME	GENDER	BATCH	EMAIL	DEGREE
1	S001	John Doe	Michael Doe	Male	2023	john.doe_updated@email.com	BSc_CS
2	S002	Jane Smith	James Smith	Female	2023	jane.smith@email.com	BSc_CS
3	S003	Sam Brown	William Brown	Male	2023	sam.brown@email.com	BSc_Physics
4	S004	Emily White	Robert White	Female	2023	emily.white@email.com	BSc_Chemistry
5	S005	Michael Green	Joseph Green	Male	2023	michael.green@email.com	BSc_Biology
6	S006	Olivia Black	Thomas Black	Female	2023	olivia.black@email.com	BSc_Math
7	S007	Daniel Harris	Edward Harris	Male	2023	daniel.harris@email.com	BSc_CS
8	S008	Sophia Clark	David Clark	Female	2023	sophia.clark@email.com	BSc_CS
9	S009	David Lewis	Paul Lewis	Male	2023	david.lewis@email.com	BSc_Physics
10	S010	Emma Turner	Brian Turner	Female	2023	emma.turner@email.com	BSc_Chemistry
11	S011	James Doe	John Doe	Male	2023	james.doe@email.com	BSc_CS

FACULTY TABLE:

	FACULTY_ID	FACULTY_NAME	GENDER	ROLE	DEPARTMENT_ID	EMAIL
1	F001	Dr. Alice Williams	Female	Head of Department	CS	alice.williams@email.com
2	F002	Dr. Bob Johnson	Male	Assistant Professor	CS	bob.johnson@email.com
3	F003	Dr. Carol Davis	Female	Professor	PH	carol.davis@email.com
4	F004	Dr. Daniel Lee	Male	Associate Professor	CH	daniel.lee@email.com
5	F005	Dr. Eva Moore	Female	Professor	BI	eva.moore@email.com
6	F006	Dr. Frank White	Male	Lecturer	MA	frank.white@email.com

NORMALIZATION

Unnormalized Table (UNF)

Here is the unnormalized table (violating 1NF, 2NF, and 3NF):

STUDENT_ID	STUDENT_NAME	FATHER_NAME	GENDER	BATCH	EMAIL	DEPARTMENT_ID	COURSE_CODE	COURSE_NAME	CREDIT_HOURS	SESSIONALS	FINAL_EXAM	GRADE	DEPARTMENT_NAME
S001	Alice Brown	Charles Brown	Female	2024	alice.brown@email.com, alt@email.com	EE	EE101, EE102	Circuits, Power Systems	3, 4	30, 35	45, 40	A, B	Electrical Eng.
S002	Bob Smith	Daniel Smith	Male	2023	bob.smith@email.com	CS	CS103, CS104	Algorithms, Operating Systems	4, 4	25, 20	38, 42	B+, A	Computer Science
S003	Clara Johnson	Edward Johnson	Female	2023	claraj@email.com	ME	ME201, ME202	Thermodynamics, Fluid Mech	3, 3	28, 32	40, 44	B, B+	Mechanical Eng.

Step 1: Normalize to 1NF

Objective: Ensure all attributes have atomic (indivisible) values, and there are no repeating groups.

Steps:

- Remove multi-valued fields (e.g., **EMAIL**, **COURSE_CODE**, **COURSE_NAME**, **CREDIT_HOURS**, **SESSIONALS**, **FINAL_EXAM**, **GRADE**).
- Create a new row for each course a student is enrolled in.

Transformed Table (1NF):

STUDENT_ID	STUDENT_NAME	FATHER_NAME	GENDER	BATCH	EMAIL	DEPARTMENT_ID	COURSE_CODE	COURSE_NAME	CREDIT_HOURS	SESSIONALS	FINAL_EXAM	GRADE	DEPARTMENT_NAME
S001	Alice Brown	Charles Brown	Female	2024	alice.brown@email.com	EE	EE101	Circuits	3	30	45	A	Electrical Eng.
S001	Alice Brown	Charles Brown	Female	2024	alice.brown@email.com	EE	EE102	Power Systems	4	35	40	B	Electrical Eng.
S002	Bob Smith	Daniel Smith	Male	2023	bob.smith@email.com	CS	CS103	Algorithms	4	25	38	B+	Computer Science
S002	Bob Smith	Daniel Smith	Male	2023	bob.smith@email.com	CS	CS104	Operating Systems	4	20	42	A	Computer Science
S003	Clara Johnson	Edward Johnson	Female	2023	claraj@email.com	ME	ME201	Thermodynamics	3	28	40	B	Mechanical Eng.
S003	Clara Johnson	Edward Johnson	Female	2023	claraj@email.com	ME	ME202	Fluid Mechanics	3	32	44	B+	Mechanical Eng.

Step 2: Normalize to 2NF

Objective: Remove partial dependencies by ensuring all non-key attributes depend on the whole primary key.

Steps:

- 1. Identify the composite primary key: **STUDENT_ID** and **COURSE_CODE** together uniquely identify each record.
- 2. Remove attributes that do not depend on the entire composite key (**DEPARTMENT_NAME** depends only on **DEPARTMENT_ID**).

Decomposed Tables (2NF):

Table 1: STUDENT_DETAILS

STUDENT_ID	STUDENT_NAME	FATHER_NAME	GENDER	BATCH	EMAIL
S001	Alice Brown	Charles Brown	Female	2024	alice.brown@email.com
S002	Bob Smith	Daniel Smith	Male	2023	bob.smith@email.com
S003	Clara Johnson	Edward Johnson	Female	2023	clara.j@email.com

Table 2: ENROLLMENTS

STUDENT_ID	COURSE_CODE	CREDIT_HOURS	SESSIONALS	FINAL_EXAM	GRADE
S001	EE101	3	30	45	A
S001	EE102	4	35	40	B
S002	CS103	4	25	38	B+
S002	CS104	4	20	42	A
S003	ME201	3	28	40	B
S003	ME202	3	32	44	B+

Table 3: COURSES

COURSE_CODE	COURSE_NAME	DEPARTMENT_ID
EE101	Circuits	EE
EE102	Power Systems	EE
CS103	Algorithms	CS
CS104	Operating Systems	CS
ME201	Thermodynamics	ME
ME202	Fluid Mechanics	ME

Table 4: DEPARTMENTS

DEPARTMENT_ID	DEPARTMENT_NAME
EE	Electrical Eng.
CS	Computer Science
ME	Mechanical Eng.

Step 3: Normalize to 3NF

Objective: Remove transitive dependencies by ensuring non-key attributes depend only on the primary key.

Steps:

- 1. COURSE_NAME depends on COURSE_CODE, which is already handled in Table 3.
- 2. DEPARTMENT_NAME depends on DEPARTMENT_ID, which is already handled in Table 4.
- 3. No transitive dependencies remain in the structure from 2NF.

Final 3NF Tables:

Table 1: STUDENT_DETAILS

STUDENT_ID	STUDENT_NAME	FATHER_NAME	GENDER	BATCH	EMAIL
S001	Alice Brown	Charles Brown	Female	2024	alice.brown@email.com
S002	Bob Smith	Daniel Smith	Male	2023	bob.smith@email.com
S003	Clara Johnson	Edward Johnson	Female	2023	clara.j@email.com

Table 2: ENROLLMENTS

STUDENT_ID	COURSE_CODE	CREDIT_HOURS	SESSIONALS	FINAL_EXAM	GRADE
S001	EE101	3	30	45	A
S001	EE102	4	35	40	B
S002	CS103	4	25	38	B+
S002	CS104	4	20	42	A
S003	ME201	3	28	40	B
S003	ME202	3	32	44	B+

Table 3: COURSES

COURSE_CODE	COURSE_NAME	DEPARTMENT_ID
EE101	Circuits	EE
EE102	Power Systems	EE
CS103	Algorithms	CS
CS104	Operating Systems	CS
ME201	Thermodynamics	ME
ME202	Fluid Mechanics	ME

Table 4: DEPARTMENTS

DEPARTMENT_ID	DEPARTMENT_NAME
EE	Electrical Eng.
CS	Computer Science
ME	Mechanical Eng.

Summary of Normalization Process:

1. **Unnormalized Table (UNF):** Contains multi-valued attributes, repeating groups, and transitive dependencies.
2. **1NF:** Removed multi-valued attributes by creating atomic columns.
3. **2NF:** Removed partial dependencies by decomposing into multiple tables.
4. **3NF:** Removed transitive dependencies by introducing separate tables for dependent attributes.